University of Piraeus
School of Information and Telecommunications Technologies
Department of Digital Systems

Level: Postgraduate Program of Study

Network Security

Case 3

Supervisor: Prof. Christos Xenakis

Kalaitzidis Ioannis ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
04/11/2025

Abstract

Implementation of a man-in-the-middle attack with an attacker (Kali Linux) and a victim (Ubuntu). The attacker is placed between the victim and the gateway through arpspoofing to monitor the connection. Using the packet sniffer from Case 2 to record the traffic in a .pcap file. Analyze the recording with Wireshark to draw conclusions.

Contents

Introduction

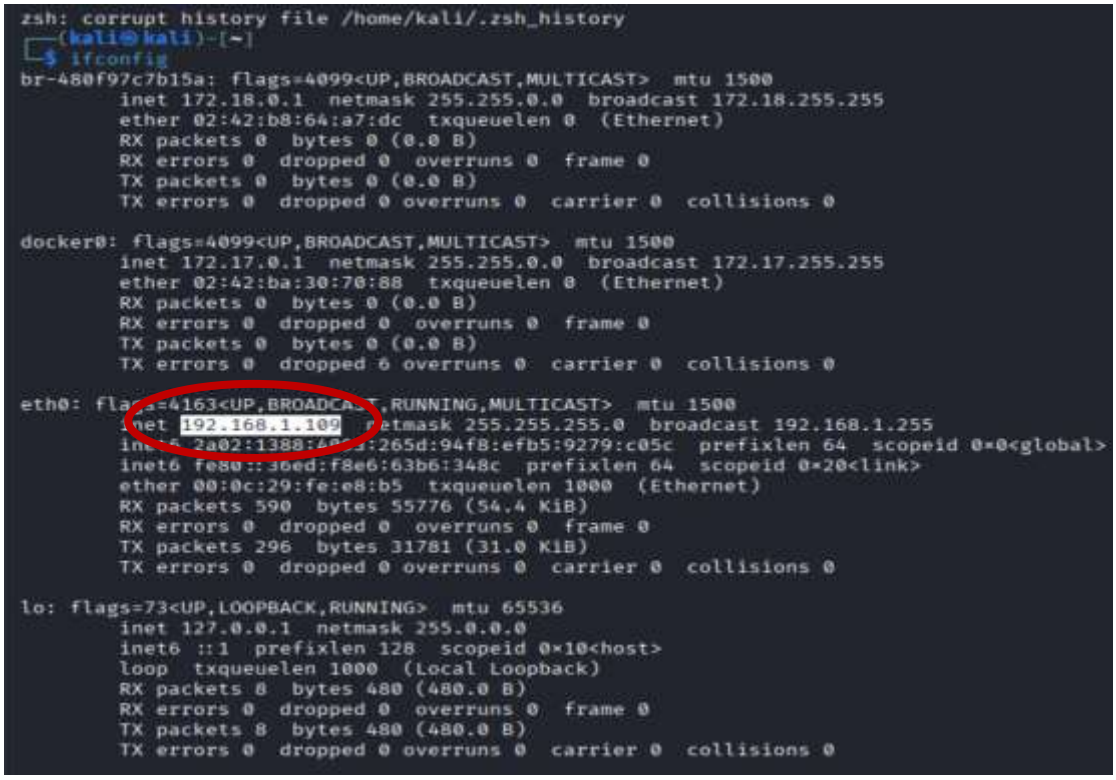
For this exercise, which is a continuation of the previous one, we will need to use one more operating system in VMware. We will choose Ubuntu 24.04.3 as the victim and Kali Linux as the attacker. In both operating systems we need to have the same type of Bridge network interface so that they are on the same network and can communicate with each other. In addition, the NAT network adapter provides the IPs to the virtual machines through the host's virtual network, while in Bridge mode the IPs are given directly by the router. Although at first we had started with NAT, we noticed that the victim's connection to the gateway/virtual router was interrupted immediately after using the arpspoof and for this reason we did the exercise with Bridge again.

Practical piece of work

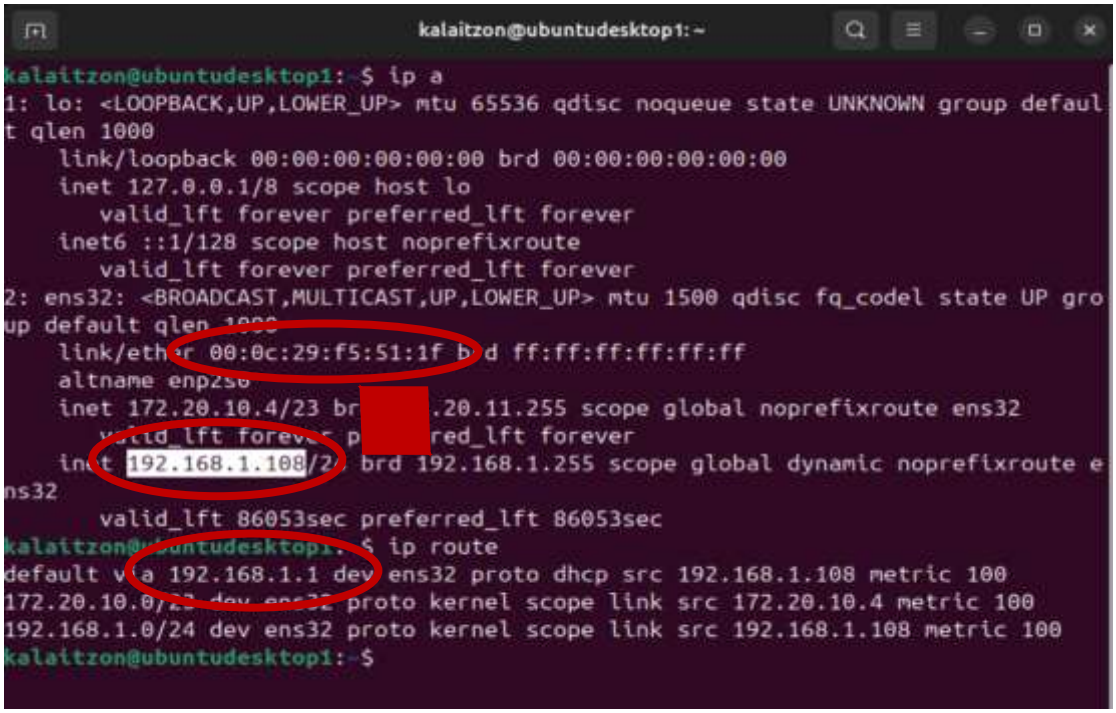
Introduction:

First we will locate the IP addresses on both systems and then, with the nmap tool, we will confirm that the Ubuntu IP corresponds to the victim. Then, with the arpspoof tool, we will position ourselves as a man-in-the-middle in the communication between Ubuntu and router: arpspoof will send false ARP replies so that the victim "sees" the MAC Address of the gateway as the MAC Address of Kali Linux and the gateway "sees" the MAC Address of the victim as the MAC Address of Kali Linux. We intend not to interrupt the connection between the victim and the router, but simply to monitor the traffic exchanged between them. After completing the process with the arpspoof, we will activate the promiscuous mode and run the sniffer from Case 2 to record the packets in a .pcap file. After the recording is over, we will open the .pcap file with the Wireshark tool to analyze and draw conclusions.

Initiating Procedures:

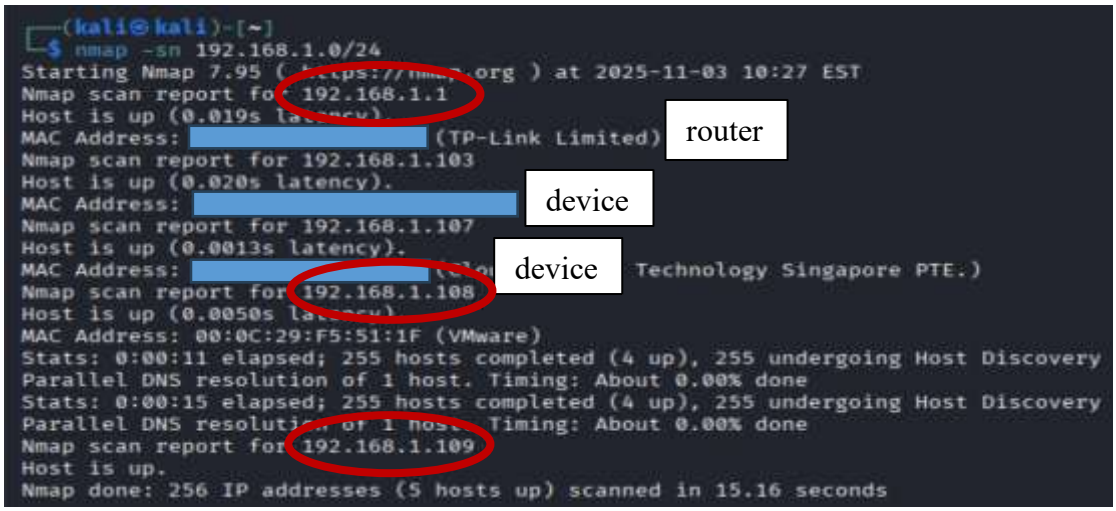


We notice that the attacker's IP (Kali Linux) is 192.168.1.109 with the **ifconfig command**.

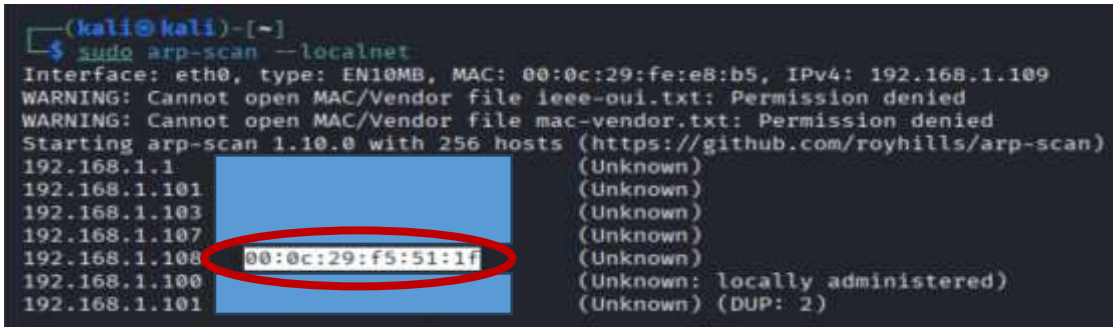


We also notice in Ubuntu (victim) we notice that its IP is 192.168.1.108 with the **ip command a** and that the IP of getaway/router is 192.168.1.1 with the **ip route command**.

Now with the nmap tool we will scan the subnet and see which IPs are active (among which is that of the victim). This will be done with the command **sudo nmap -sn 192.168.1.0/24**. The reason we put /24 is to "catch" every active IP of the network mask (starting from .0 to .254) and show them to us.

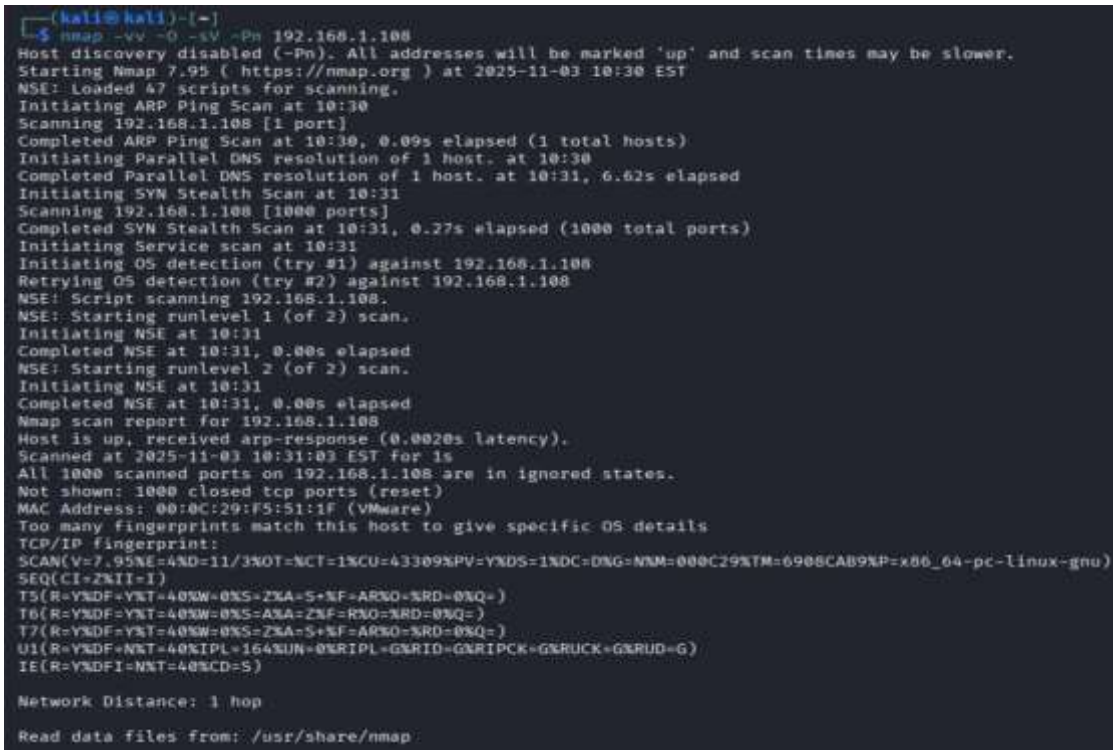


From the scan (nmap -sn 192.168.1.0/24) we notice that the address **192.168.1.1** is the **gateway/router**. The address **192.168.1.109** corresponds to the attacking machine (Kali Linux) as we know from ifconfig, while the **192.168.1.108** is the victim's (Ubuntu) because the rest of the IPs are from my devices. So we know for sure the IP of the victim. However, for confirmation we can also cross-check it from MAC Address. With the **sudo arp-scan --localnet** command we can see which IP address corresponds to its own MAC Address. We can cross-check this from the ip a command that we used above in Ubuntu.



Under other circumstances this still doesn't hold because as an attacker I don't know anything about the OS. With the command **nmap -vv -O -sV -Pn 192.168.1.108** (where -vv for verbose, -O to locate the OS, -sV to locate a service, and -Pn not to ping the specific address if we know it's active) I would normally get the information that the OS is Ubuntu. Most recent with the latest updates, NMAP is not properly identified and we are not given this necessary information as shown in the image below. The only

thing that confirms us is that this particular machine with this IP is OS but it cannot recognize anything more (Too many fingerprints match this host to give specific OS details). However, in the event that we know the other IPs (that they belong to other devices) as now, it is enough.



We then pinged each IP address to verify the basic connectivity (reachability) before proceeding with arpspoofing from Kali Linux. With this we confirm that the IP addresses are up and that there is a route to each IP.

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.1.108
PING 192.168.1.108 (192.168.1.108) 56(84) bytes of data.
64 bytes from 192.168.1.108: icmp_seq=1 ttl=64 time=6.91 ms
64 bytes from 192.168.1.108: icmp_seq=2 ttl=64 time=1.93 ms
64 bytes from 192.168.1.108: icmp_seq=3 ttl=64 time=1.72 ms
64 bytes from 192.168.1.108: icmp_seq=4 ttl=64 time=1.61 ms
^C
── 192.168.1.108 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 1.608/3.040/6.910/2.236 ms

┌──(kali㉿kali)-[~]
└─$ ping 192.168.1.109
PING 192.168.1.109 (192.168.1.109) 56(84) bytes of data.
64 bytes from 192.168.1.109: icmp_seq=1 ttl=64 time=0.126 ms
64 bytes from 192.168.1.109: icmp_seq=2 ttl=64 time=0.131 ms
64 bytes from 192.168.1.109: icmp_seq=3 ttl=64 time=0.123 ms
^C
── 192.168.1.109 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2039ms
rtt min/avg/max/mdev = 0.123/0.126/0.131/0.003 ms

┌──(kali㉿kali)-[~]
└─$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=22.1 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=11.7 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=8.37 ms
^C
── 192.168.1.1 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 8.368/14.029/22.052/5.830 ms
```

We did the same from Ubuntu as shown in the image below.

```
kalaitzon@ubuntudesktop1:~$ ping 192.168.1.109
PING 192.168.1.109 (192.168.1.109) 56(84) bytes of data.
64 bytes from 192.168.1.109: icmp_seq=1 ttl=64 time=1.24 ms
64 bytes from 192.168.1.109: icmp_seq=2 ttl=64 time=4.38 ms
64 bytes from 192.168.1.109: icmp_seq=3 ttl=64 time=1.96 ms
^C
--- 192.168.1.109 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.242/2.525/4.379/1.342 ms
kalaitzon@ubuntudesktop1:~$ ping 192.168.1.108
PING 192.168.1.108 (192.168.1.108) 56(84) bytes of data.
64 bytes from 192.168.1.108: icmp_seq=1 ttl=64 time=0.068 ms
64 bytes from 192.168.1.108: icmp_seq=2 ttl=64 time=0.108 ms
64 bytes from 192.168.1.108: icmp_seq=3 ttl=64 time=0.110 ms
^C
--- 192.168.1.108 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2069ms
rtt min/avg/max/mdev = 0.068/0.095/0.110/0.019 ms
kalaitzon@ubuntudesktop1:~$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=91.3 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=7.74 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=8.61 ms
^C
--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 7.738/35.890/91.327/39.201 ms
kalaitzon@ubuntudesktop1:~$
```

```
┌──(kali㉿kali)-[~]
└─$ sudo ip link set dev eth0 promisc on
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
```

We activate the promiscuous mode with the command **sudo ip link set dev eth0 promisc on** and control it with the command **ip link show eth0** to receive all the traffic of the interface and analyze packets.

```
┌──(kali㉿kali)-[~]
└─$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
1

┌──(kali㉿kali)-[~]
└─$ cat /proc/sys/net/ipv4/ip_forward
1
```

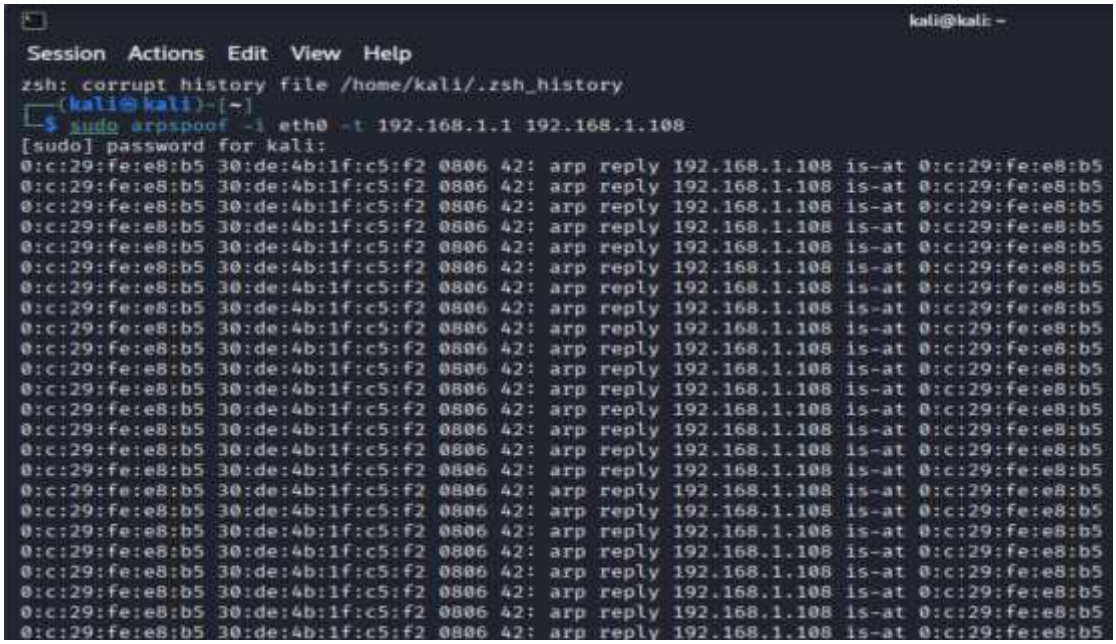
Next we will write the value 1 in the system file /proc/sys/net/ipv4/ip_forward (with the command **echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward).** In a MITM and arpspoof scenario, if the attacker (Kali Linux) "takes" the traffic between the victim and the gateway, it must transmit the packets to the actual gateway so that the victim can continue to have Internet. Without IP forwarding, the attacker would accept the packets but would not forward them, causing the victim to lose connection. and then we checked the value with cat /proc/sys/net/ipv4/ip_forward, which returned 1, which means that the net.ipv4.ip_forward parameter is enabled. Then we will check the specific value with the command **cat /proc/sys/net/ipv4/ip_forward**. Since it returns the value 1, it has been activated.

The above only concerns IPv4 because arpspoof works through the ARP (Address Resolution Protocol) protocol and affects IPv4 traffic exclusively. ARP is a protocol that translates an IPv4 address into a physical network address. It is used so that a machine knows which MAC Address to send the packet to when it wants to communicate with an IP on the same local area network (LAN). IPv6 does not use ARP but the NDP (Neighbor Discovery Protocol) protocol. So IPv6 packets are not affected by arpspoof. In practice, this means that in a dual-stack environment (IPv4+IPv6) modern browsers and services tend to test IPv6 first, and if there is a valid IPv6 path, traffic "escapes" the arpspoof. For this reason, we noticed a delay in the "opening" of

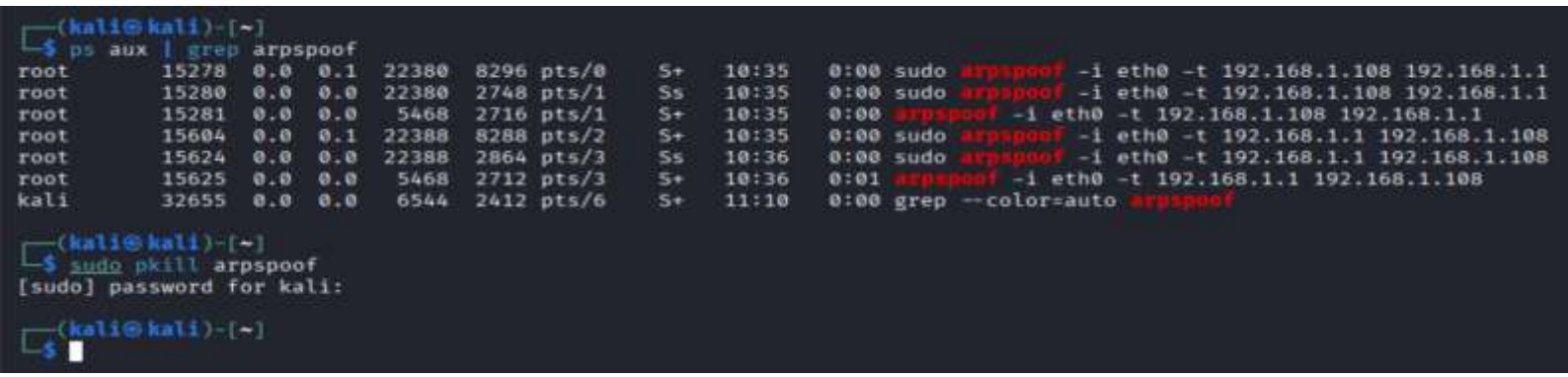
websites. In other words, it tried via IPv6, delayed and returned to IPv4 format or vice versa (usually this). However, a prerequisite, which we did not know until we realized what was happening, was that IPv6 is definitely active and not disabled as we had it in the beginning. With IPv6 disabled, websites and services loaded and at the end they displayed the message "Offline". This was because, as mentioned above, most websites and services support IPv6 (or at least the ones we tested) and could not load via IPv4, which was the only enabled protocol. As soon as we enabled IPv6 the web pages loaded normally.

Maybe with another tool like bettercap that supports ARP and NDP protocols or mitm6, especially for IPv6/DNS takeover, the MITM script would be made easier.

Now we will use the arpspoof tool to start the MITM monitoring of the victim's communication (Ubuntu) and getaway. Specifically with the command **sudo arpspoof -i eth0 -t 192.168.1.108 192.168.1.1** where we practically tell the victim (Ubuntu) that I (Kali Linux) am the gateway. In other words, the command sends ARP replies to the target 192.168.1.108, falsely telling it that IP 192.168.1.1 (the gateway) has the MAC Address of the attacker (Kali Linux).



We continue to a separate terminal with the command **sudo arpspoof -i eth0 -t 192.168.1.1 192.168.1.108** where we practically tell the gateway that I am the victim (Ubuntu). In other words, respectively, the command tells the gateway that the IP of the victim (192.168.1.108) has the MAC of the attacker (Kali Linux). Thus, both ends (victim and gateway) send their packets to the attacker and as a result we have MITM.



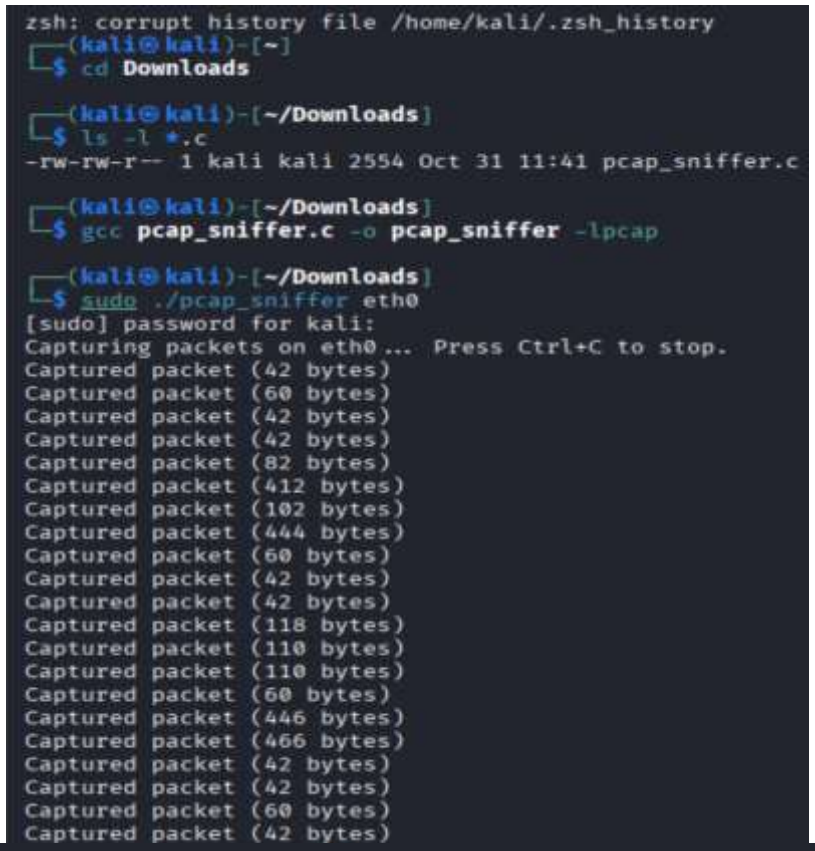
Useful commands for arpspoof:

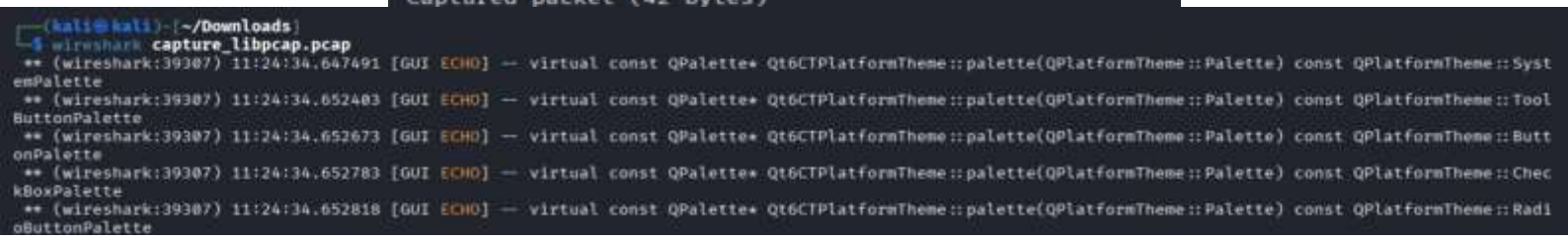
sudo pkill arpspoof | Closes all ARPSPOOF processes

ps aux | grep arpspoof | checks if there are still processes with this name

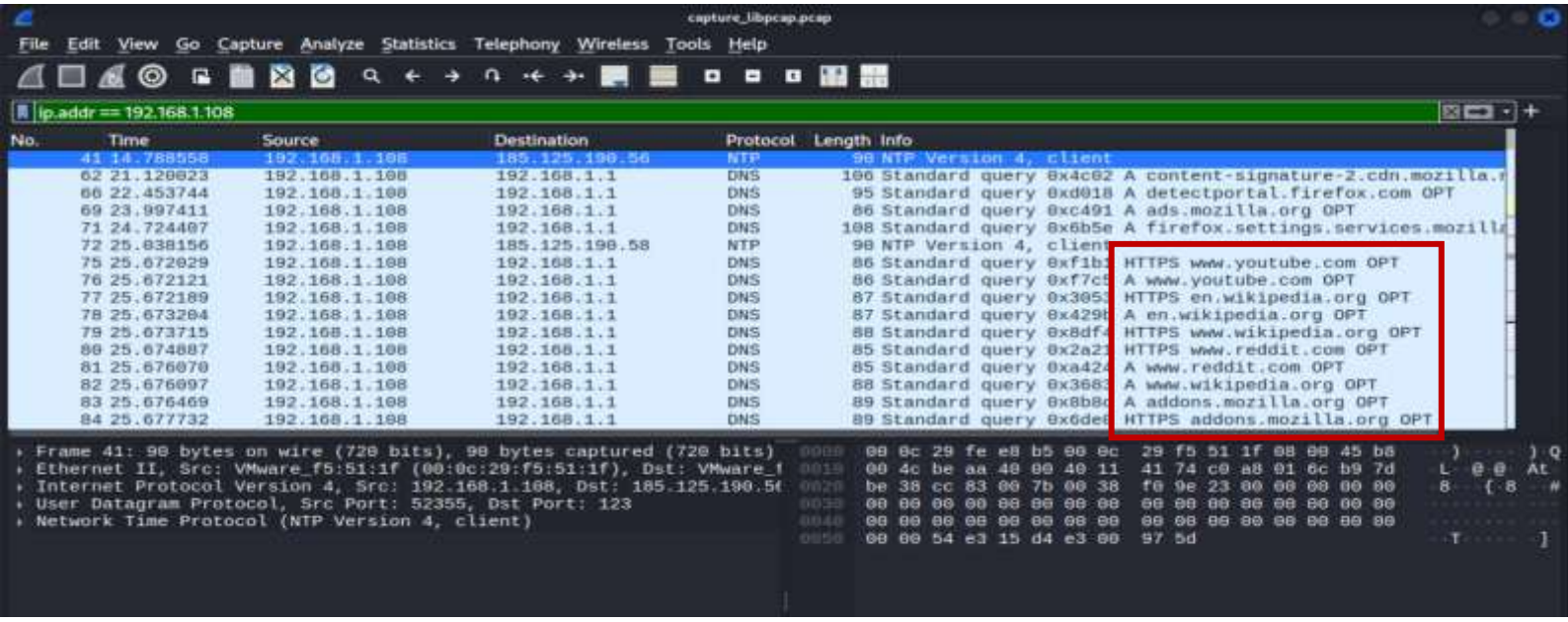
Then we compile the packet sniffer located in the Downloads folder. After the compilation is completed (**gcc pcap_sniffer.c -o pcap_sniffer -lpcap**), we will run the sniffer in the interface eth0 as we did in Case 2. The recording of the packages will start and will stop as soon as we select the Ctrl + C buttons on the keyboard. The packages

will be saved in the .pcap file which we will open through wireshark to export the results.

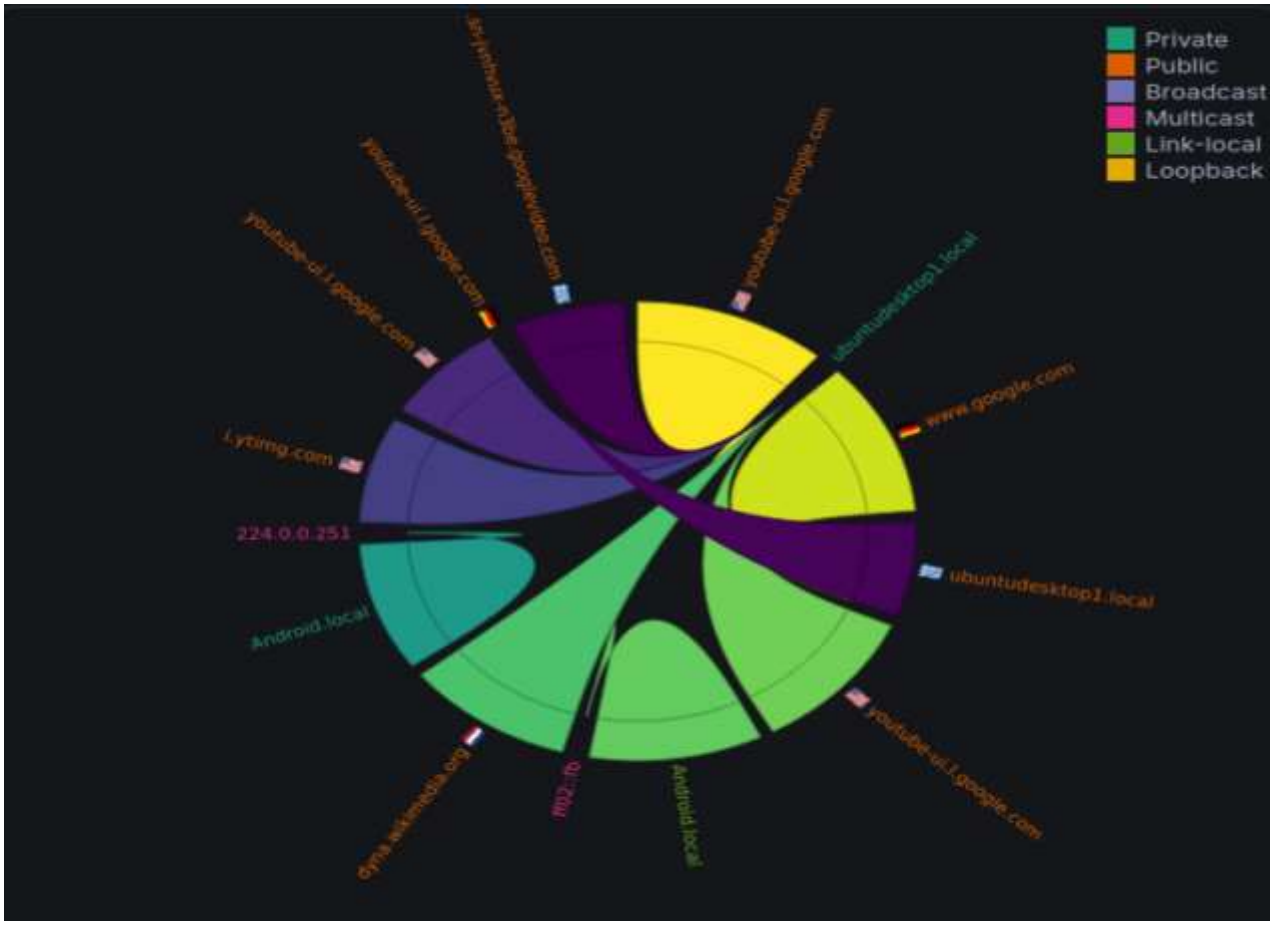




By opening the capture_libpcap.pcap file with the command (**wireshark capture_libpcap.pcap)** with wireshark we detected and recorded the packet traffic between the victim (Ubuntu) and the gateway. In capture, packets and requests to various websites and services visited during the test (e.g. Wikipedia, YouTube, Reddit) are displayed, confirming the successful recording of the network activity.



The following image, taken from the analysis of the .pcap file, successfully records the victim's online activity (Ubuntu): calls to Wikipedia and YouTube appear as conversations to the respective domains/hosts.



Source: https://apackets.com/

Conclusion:

The exercise demonstrated in practice how a Man in the Middle (MITM) attack can be carried out using the arpspoof tool, which misleads the victim and the getaway so that their communication passes through the attacker. By enabling IP forwarding and simultaneously recording network traffic with Wireshark, it was possible to monitor the packets exchanged between them. Overall, the exercise highlighted the functionality of the arpspoof tool and confirmed the capability of intercepting and monitoring packets at the local area network (LAN) level.

Bibliography

danscourses. (2012, March 27). *ARP Spoofing - Man-in-the-middle attack* [Video file]. Retrieved from https://www.youtube.com/watch?v=hI9J_tnNDCc

Narten, T., Nordmark, E., & Simpson, W. (1998). *Neighbor Discovery for IP Version 6 (IPV6)*. https://doi.org/10.17487/rfc2461

Introduction. (n.d.). Bettercap. https://www.bettercap.org/project/introduction/

Tools of the Trade: IPv6 DNS Takeover with MitM6. (2023, September 22). https://www.evolvesecurity.com/blog-posts/tools-of-the-trade-ipv6-dns-takeover-with-mitm6

Kampourakis, V., Kambourakis, G., Chatzoglou, E., & Zaroliagis, C. (2022). Revisiting man-in-the-middle attacks against HTTPS. *Network Security*, *2022*(3). https://doi.org/10.12968/s1353-4858(22)70028-1

Vaishnavi. (2025, June 19). How to Use Bettercap for Network Penetration Testing ? Beginner's Guide with Commands and Use Cases. *WebAsha Technologies*. https://www.webasha.com/blog/how-to-use-bettercap-for-network-penetration-testing-beginners-guide-with-commands-and-use-cases

GeeksforGeeks. (2025, October 10). *Address Resolution Protocol ARP*. GeeksforGeeks. https://www.geeksforgeeks.org/computer-networks/arp-protocol/

ChatGPT. (n.d.). Retrieved from https://chatgpt.com/

Kumar, G. (2025, July 15). *IPV6 Neighbor Discovery Protocol (NDP) explained*. https://www.uninets.com/UniNets. https://www.uninets.com/blog/neighbor-discovery-protocol

Infosec. (2021, April 27). *How to set up a man in the middle attack | Free Cyber Work Applied series* [Video]. YouTube. https://www.youtube.com/watch?v=GkexkyUbUd4

Tech Sky - Ethical Hacking. (2024, September 15). *How to Spy on Any Device's Network using ARP Spoofing in Kali Linux?* [Video]. YouTube. https://www.youtube.com/watch?v=YtLsI-wHv2U

CertBros. (2021, April 6). *ARP Poisoning | Man-in-the-Middle attack* [Video]. YouTube. https://www.youtube.com/watch?v=A7nih6SANYs

Chris Greer. (2021, December 9). *How ARP poisoning works // Man-in-the-Middle* [Video]. YouTube. https://www.youtube.com/watch?v=cVTUeEoJgEg